

Cloud Computing Lab

Experiment 3 Report

Student name	Deep Shekhar Halder
Roll number	UG/02/BTCSE/2023/063
Course	Cloud Computing Theory & Lab
Instructor	Anusuya Bera, Assistant Professor (CSE, SOET)
Institution	Adamas University, Kolkata
Experiment No.	3
Title	Creation of a Simple Network Topology Using Open-Source Network Virtualization Tools

1. Objective

To create and configure a simple network topology using an open-source network virtualization tool (Mininet) and verify communication between the virtual network nodes.

2. Requirements

- Ubuntu Linux Virtual Machine (Ubuntu-CloudLab)
- Oracle VirtualBox
- Mininet (network emulator)

3. Task

- Create network topologies of two types (single and tree), each with two different node counts.
- View links and verify connectivity using basic Linux networking commands.
- Stop the network cleanly after each test.

4. Procedure and Observations

4.1 Installing Mininet

The Ubuntu-CloudLab VM's package list was updated and Mininet was installed using apt.

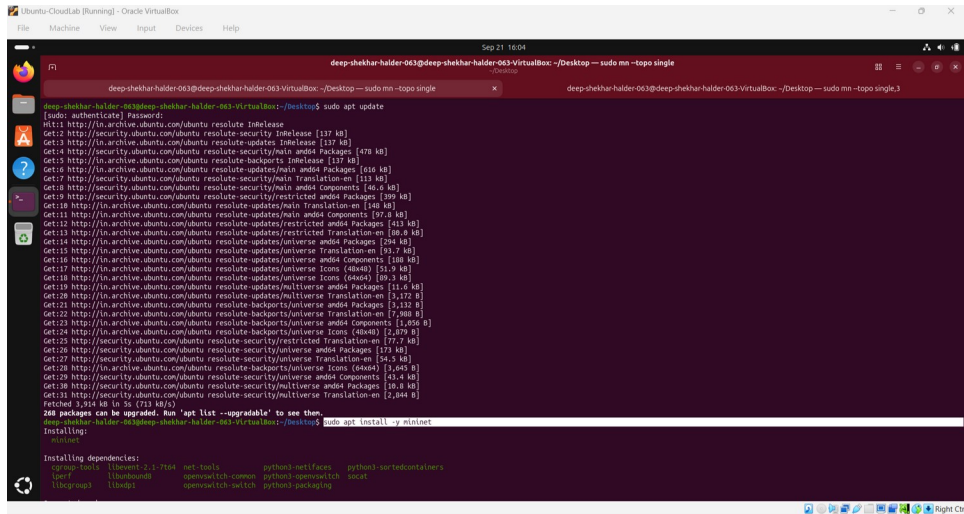


Fig 1: Updating the package list and starting Mininet installation (sudo apt update && sudo apt install -y mininet)

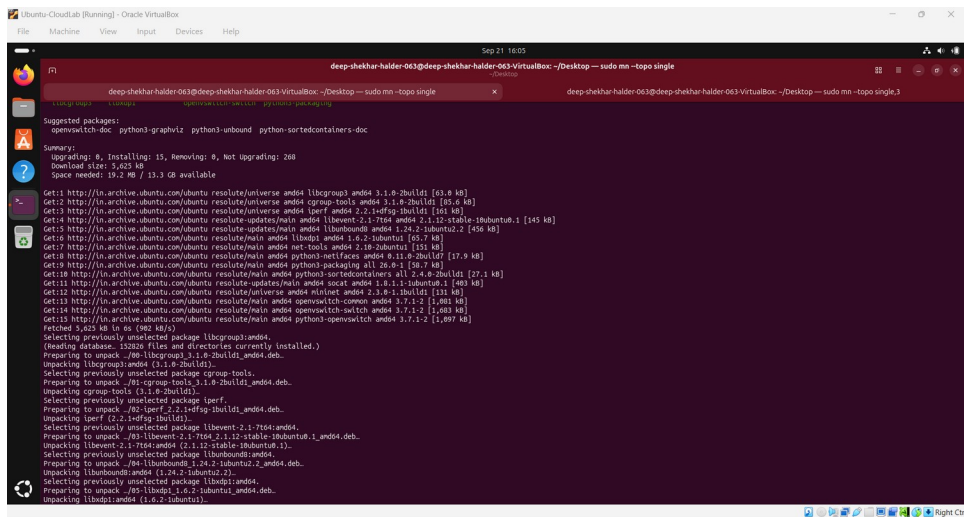


Fig 2: Installing Mininet and its dependencies (openvswitch, python3-openvswitch, etc.)

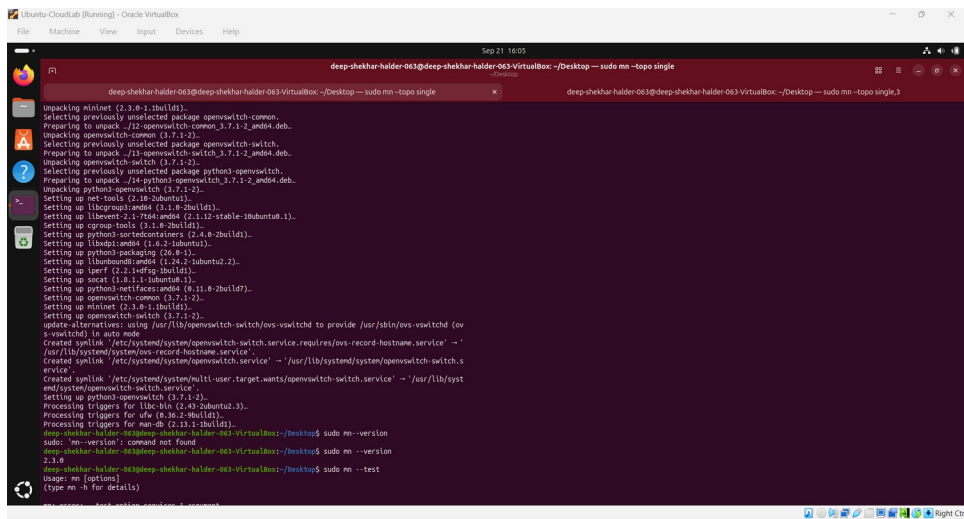
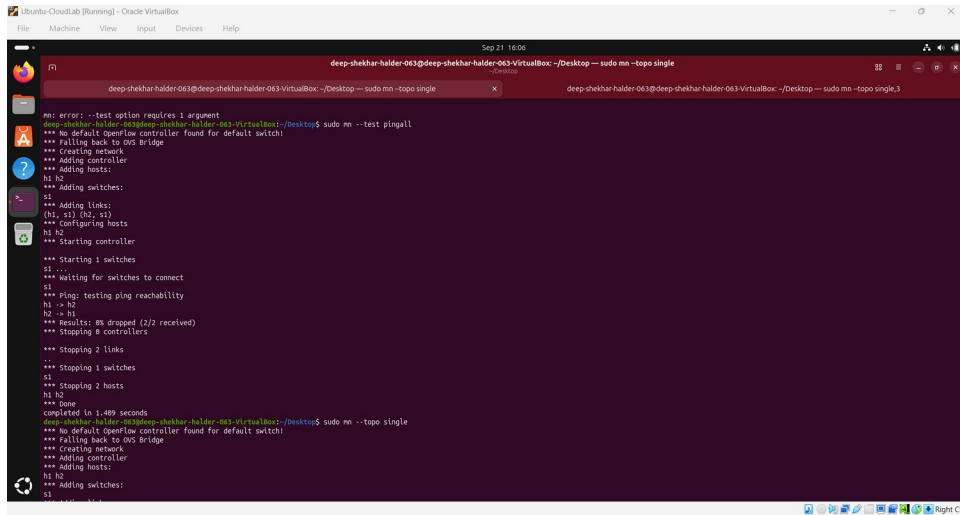


Fig 3: Verifying the installed Mininet version (`sudo mn --version -> 2.3.0`)

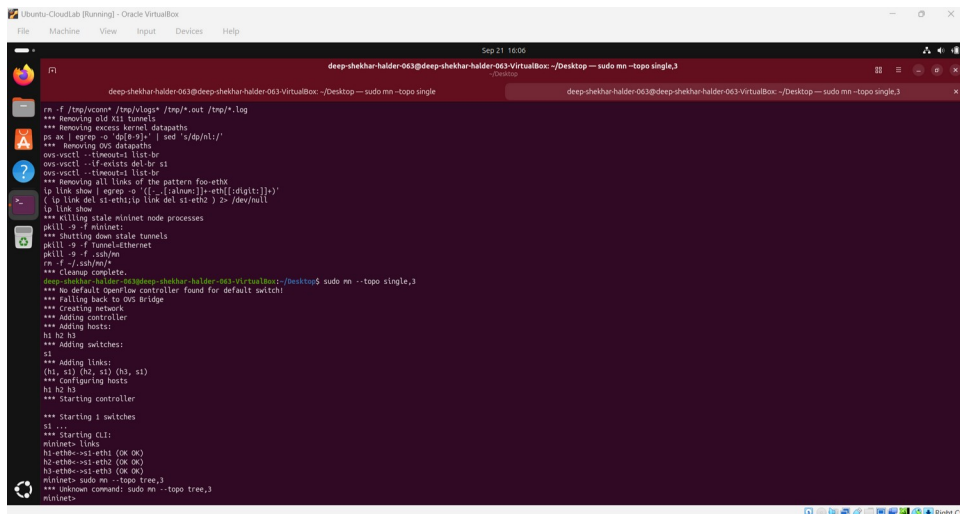
4.2 Initial Connectivity Test

Before building custom topologies, Mininet's built-in self-test was used to confirm the network stack was functioning correctly.



```
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop -- sudo mn --topo single
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop -- sudo mn --topo single
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop -- sudo mn --topo single
mn: error: --test option requires 1 argument
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop$ sudo mn --test pingall
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
*** Starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Ping: testing ping reachability
h1 -> h2
*** Results: 8s dropped (2/2 received)
*** Stopping 0 controllers
*** Stopping 2 links
s1 ...
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 1.489 seconds
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop$ sudo mn --topo single
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
```

Fig 4: Running the built-in pingall self-test (`sudo mn --test pingall`) and starting a single topology

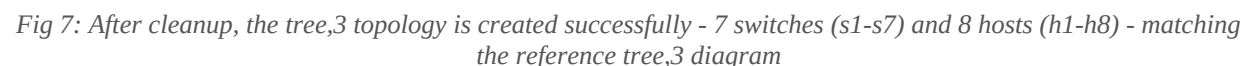


```
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop -- sudo mn --topo single,3
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop -- sudo mn --topo single,3
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop -- sudo mn --topo single,3
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*_net /tmp/*_log
*** Removing old xli tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]*' | sed 's/dp/nli/'
*** Removing ovs datapaths
ovs-vsctl --if-exists del-br s1
ovs-vsctl --if-exists del-br s2
*** Removing all links of the pattern foo-eth*
ip link show | egrep -o '([a-zm0-9]*)-eth([0-9]{1,3})'
{ ip link del s1-eth1 ip link del s1-eth2 } 2> /dev/null
ip link show
*** Killing stale mininet node processes
pkill -9 -f Mininet
*** Shutting down stale tunnels
pkill -9 -f Tunnel ethernet
pkill -9 -f ssh/m
rm -f ~/.ssh/m/*
*** Cleanup complete.
deep-shekar-halder-063@deep-shekar-halder-063-VirtualBox: ~/Desktop$ sudo mn --topo single,3
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> links
h1-eth0-s1-eth1 (OK OK)
h2-eth0-s1-eth2 (OK OK)
h3-eth0-s1-eth3 (OK OK)
mininet> sudo mn --topo tree,3
*** Unknown command: sudo mn --topo tree,3
mininet>
```

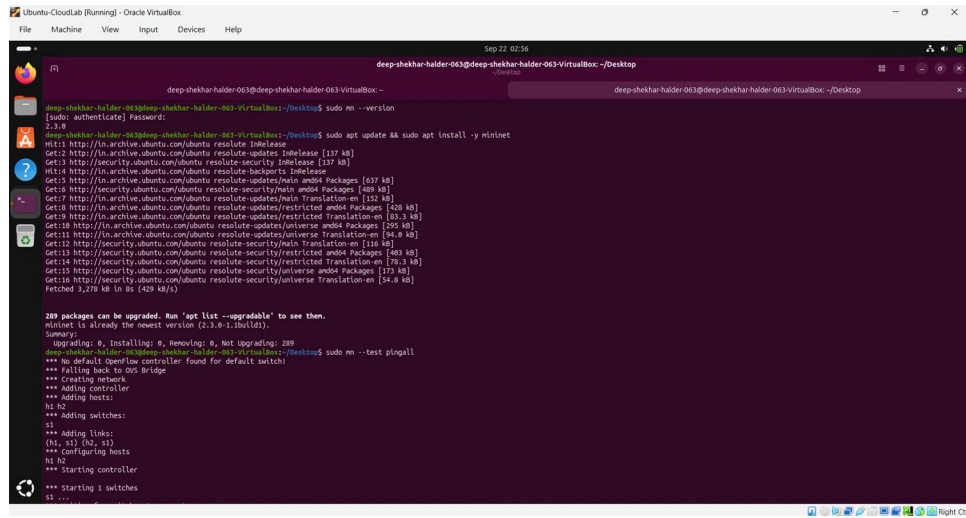
Fig 5: Viewing switch-host links inside the Mininet CLI. Note: '`sudo mn --topo tree,3`' was mistakenly typed at the `mininet>` prompt instead of the shell, producing an 'Unknown command' error.

4.3 Building the tree,3 Topology

After exiting the Mininet CLI correctly, the `tree,3` command was re-attempted from the shell. The first attempt failed with an `RTNETLINK` exception because leftover virtual interfaces from the previous session had not been fully cleaned up; Mininet's automatic cleanup routine then cleared the stale interfaces before the topology was rebuilt.

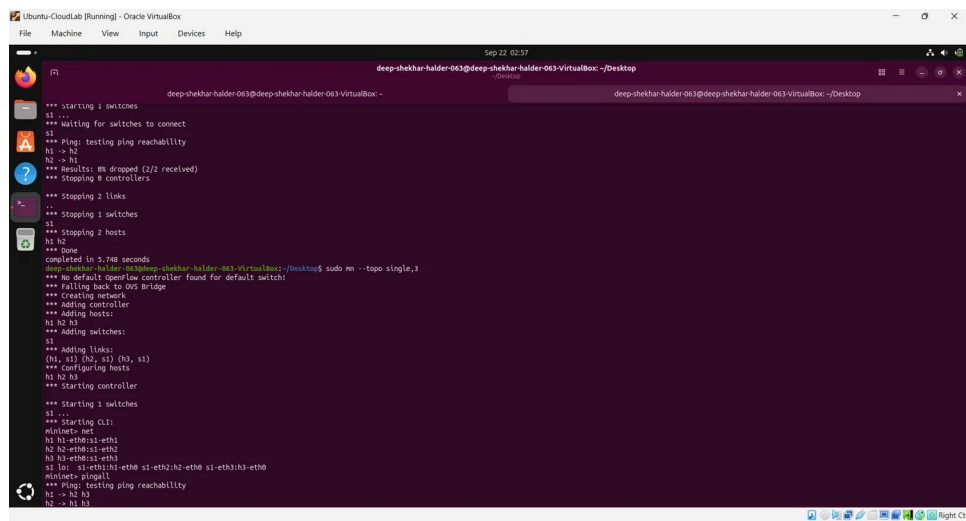


4.4 Re-verifying the Topologies



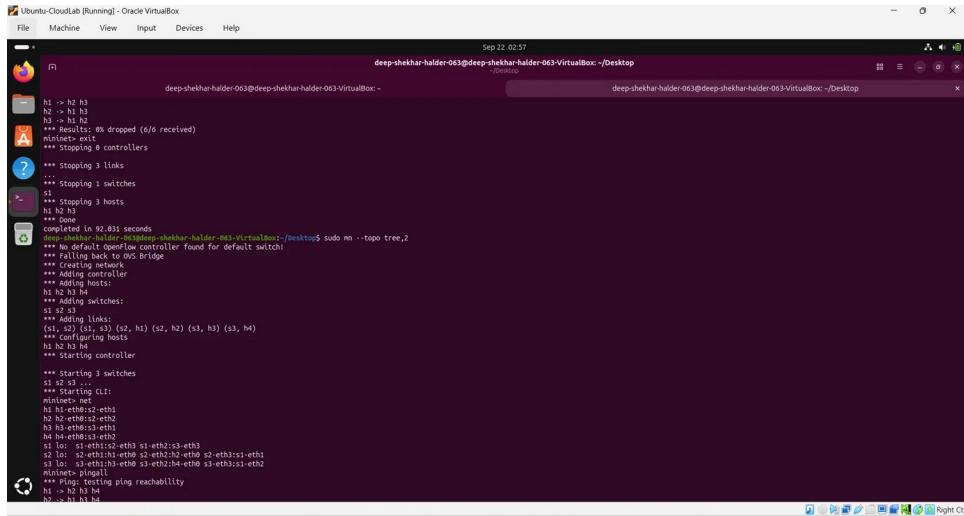
```
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox: ~/Desktop
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox:~/Desktop$ sudo mn --version
2.3.0
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox:~/Desktop$ sudo apt update && sudo apt install -y mininet
Hit:1 http://in.archive.ubuntu.com/ubuntu resolute InRelease [137 kB]
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:4 http://in.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:3 http://in.archive.ubuntu.com/ubuntu resolute-updates InRelease [687 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [489 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [152 kB]
Get:8 http://in.archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [528 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [83.3 kB]
Get:10 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [509 kB]
Get:11 http://in.archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [34.8 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [116 kB]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [483 kB]
Get:14 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [79.3 kB]
Get:15 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [172 kB]
Get:16 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [54.8 kB]
Fetched 3,278 kB in 8s (429 kB/s)
289 packages can be upgraded. Run 'apt list --upgradable' to see them.
Mininet is already the newest version (2.3.0-1.0ubuntu1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 289
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox:~/Desktop$ sudo mn --test pingall
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
  h1 h2
*** Adding switches:
  s1
*** Adding links:
  (h1, s1) (h2, s1)
*** Configuring hosts
  h1 h2
*** Starting controller
*** Starting 1 switches
  s1 ...
```

Fig 8: Confirming Mininet is still installed (mn --version -> 2.3.0) and running the pingall self-test



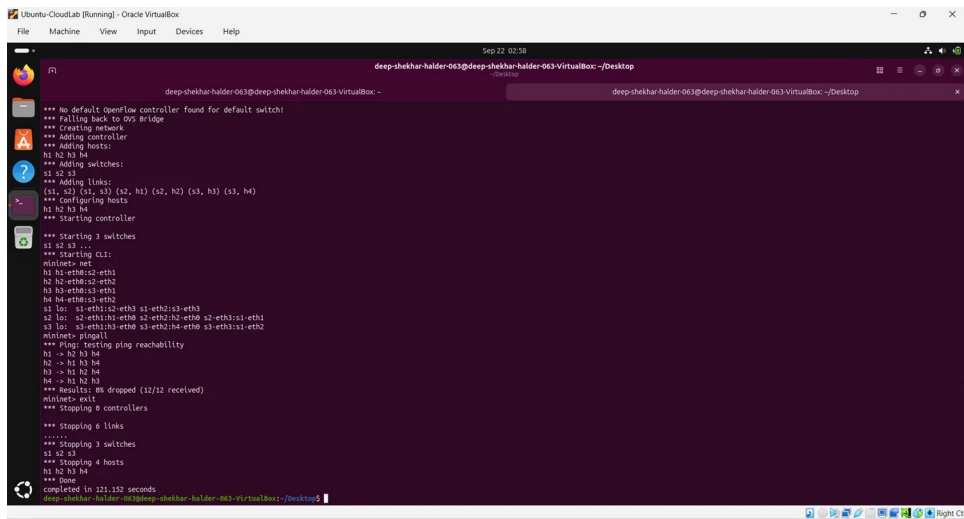
```
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox: ~/Desktop
*** starting 1 switches
s1 ...
*** Waiting for switches to connect
s1
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (1/2 received)
*** Stopping 0 controllers
*** Stopping 2 links
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 5.748 seconds
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox:~/Desktop$ sudo mn --topo single,3
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
  h1 h2 h3
*** Adding switches:
  s1
*** Adding links:
  (h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
  h1 h2 h3
*** Starting controller
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> net
h1 h1-eth0:si-eth1
h2 h2-eth0:si-eth2
h3 h3-eth0:si-eth3
s1 lo: si-eth1:h1-eth0 si-eth2:h2-eth0 si-eth3:h3-eth0
mininet> pingall
*** Ping: testing ping reachability
h1 -> h1 h3
h2 -> h1 h3
```

Fig 9: Creating the single,3 topology and listing its links with 'net'



```
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox: ~  
h1 -> h2 h3  
h2 -> h1 h3  
h3 -> h1 h2  
*** Results: 0% dropped (6/6 received)  
mininet> exit  
*** Stopping 6 controllers  
...  
*** Stopping 3 links  
...  
*** Stopping 1 switches  
s1  
*** Stopping 3 hosts  
h1 h2 h3  
*** Done  
completed in 92.611 seconds  
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox: ~/Desktop$ sudo mn --topo tree,2  
*** No default OpenFlow controller found for default switch!  
*** Falling back to OVS Bridge  
*** Creating network  
*** Adding controller  
*** Adding hosts:  
h1 h2 h3 h4  
*** Adding switches:  
s1 s2 s3  
*** Adding links:  
(s1, s2) (s1, s3) (s2, h1) (s2, h2) (s3, h3) (s3, h4)  
*** Configuring hosts  
h1 h2 h3 h4  
*** Starting controller  
...  
*** Starting 3 switches  
s1 s2 s3 ...  
*** Starting CLI:  
mininet> net  
h1 h1-eth0:s2-eth1  
h2 h2-eth0:s2-eth2  
h3 h3-eth0:s3-eth1  
h4 h4-eth0:s3-eth2  
s1 lo: s1-eth1:s2-eth3 s1-eth2:s3-eth3  
s2 lo: s2-eth1:h1-eth0 s2-eth2:h2-eth0 s2-eth3:s1-eth1  
s3 lo: s3-eth1:h3-eth0 s3-eth2:h4-eth0 s3-eth3:s1-eth2  
mininet> pingall  
*** Ping: testing ping reachability  
h1 -> h2 h3 h4  
h2 -> h1 h3 h4
```

Fig 10: Verifying single,3 connectivity with pingall (6/6 received, 0% dropped), then creating the tree,2 topology



```
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox: ~  
*** No default OpenFlow controller found for default switch!  
*** Falling back to OVS Bridge  
*** Creating network  
*** Adding controller  
*** Adding hosts:  
h1 h2 h3 h4  
*** Adding switches:  
s1 s2 s3  
*** Adding links:  
(s1, s2) (s1, s3) (s2, h1) (s2, h2) (s3, h3) (s3, h4)  
*** Configuring hosts  
h1 h2 h3 h4  
*** Starting controller  
...  
*** Starting 3 switches  
s1 s2 s3 ...  
*** Starting CLI:  
mininet> net  
h1 h1-eth0:s2-eth1  
h2 h2-eth0:s2-eth2  
h3 h3-eth0:s3-eth1  
h4 h4-eth0:s3-eth2  
s1 lo: s1-eth1:s2-eth3 s1-eth2:s3-eth3  
s2 lo: s2-eth1:h1-eth0 s2-eth2:h2-eth0 s2-eth3:s1-eth1  
s3 lo: s3-eth1:h3-eth0 s3-eth2:h4-eth0 s3-eth3:s1-eth2  
mininet> pingall  
*** Ping: testing ping reachability  
h1 -> h2 h3 h4  
h2 -> h1 h3 h4  
h3 -> h1 h2 h4  
h4 -> h1 h2 h3  
*** Results: 0% dropped (12/12 received)  
mininet> exit  
*** Stopping 6 controllers  
...  
*** Stopping 6 links  
...  
*** Stopping 3 switches  
s1 s2 s3  
*** Stopping 6 hosts  
h1 h2 h3 h4  
*** Done  
completed in 121.152 seconds  
deep-shekhhar-halder-063@deep-shekhhar-halder-063-VirtualBox: ~/Desktop$
```

Fig 11: Verifying tree,2 connectivity with pingall (12/12 received, 0% dropped) and stopping the network

5. Topology Diagrams

The logical structure of the topologies tested above is illustrated below.

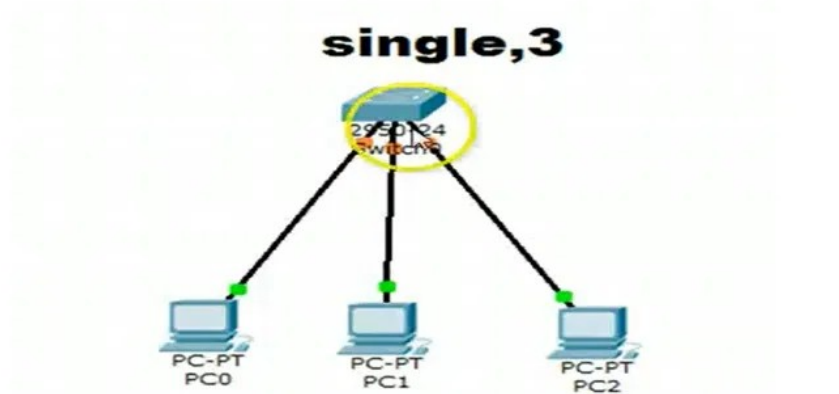


Fig 12: Logical diagram of the single,3 topology - one switch connected to three hosts

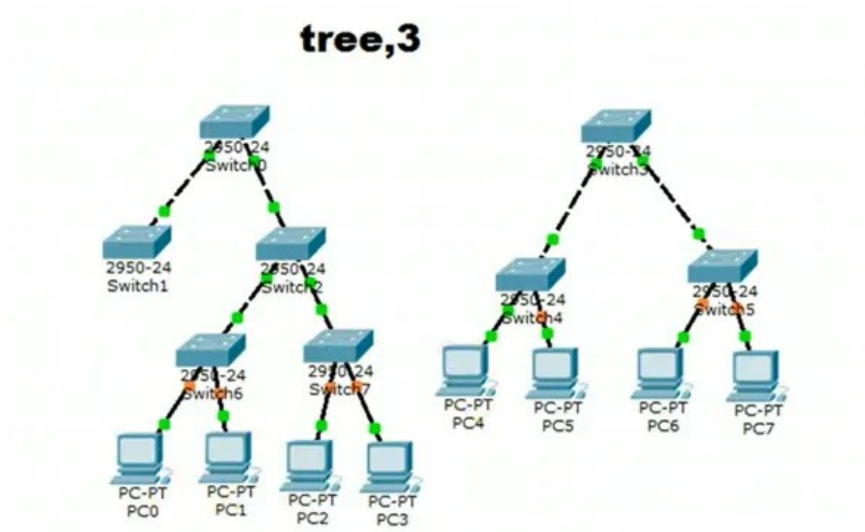


Fig 13: Logical diagram of the tree,3 topology - a hierarchical arrangement of 7 switches and 8 hosts

6. Results

Topology	Nodes	Ping result
single,3	1 switch, 3 hosts	0% dropped (6/6 received)
tree,3	7 switches, 8 hosts	Topology created successfully
tree,2	3 switches, 4 hosts	0% dropped (12/12 received)

7. Conclusion

Mininet was successfully installed and used to create three distinct network topologies on an Ubuntu virtual machine: a single-switch topology (single,3), a three-level tree topology (tree,3 - 7 switches, 8 hosts), and a smaller two-level tree (tree,2) used for re-verification. Full host-to-host connectivity was confirmed for the single and tree,2 topologies using the pingall command, with 0% packet loss in both cases. The exercise also involved resolving an

RTNETLINK interface-creation conflict caused by incompletely cleaned-up virtual interfaces from a prior run, reinforcing practical troubleshooting skills alongside the core Mininet networking concepts.